

Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Licenciatura en Ciencias de la Computación

Trabajo Práctico Sistemas Operativos I

Santino Bisutti
santibisutti2005@gmail.com

Martín Pérez de Urrechu
martinperezdeurrechu@gmail.com

Bruno Minighin
brunominighin@gmail.com

Valentino Domínguez
valentino0dominguez@gmail.com

Junio de 2026

1 Roles y contribuciones

- **Santino Bisutti** (Ingeniero de comunicaciones en C)
- **Martín Pérez de Urrechú** (Gestor de recursos y estado)
- **Valentino Domínguez** (Planificador y lógica anti-deadlock)
- **Bruno Minighin** (Planificador y lógica anti-deadlock)

Durante la fase inicial del desarrollo, creímos conveniente subdividirnos en dos grupos. Santino y Martín se encargaron del agente C y, por otro lado, Valentino y Bruno llevaron adelante la implementación del cliente Erlang. Para finalizar, entre todos los miembros del equipo, se realizó la integración, los tests y la documentación.

2 Estrategia contra deadlock

Teniendo en cuenta las restricciones de comunicación impuestas por el enunciado del trabajo, para evitar deadlocks en el sistema se nos ocurrió utilizar una estrategia de **timeouts y desincronización aleatoria**. Esto con el fin de romper la simetría de las colisiones y garantizar el progreso del sistema sin requerir comunicación adicional.

Escenario de deadlock distribuido:

Supongamos que tenemos los siguientes recursos en dos nodos:

- **Nodo A:** 2 CPUs, 8 GB de RAM y 0 GPU.
- **Nodo B:** 2 CPUs, 4 GB de RAM, 1 GPU.

Existen dos trabajos planificados concurrentemente:

- **Job1** (desde Nodo A): requiere 2 CPUs locales de A y 1 GPU remota de B.
- **Job2** (desde Nodo B): requiere 1 GPU local de B y 2 CPUs remotas de A.

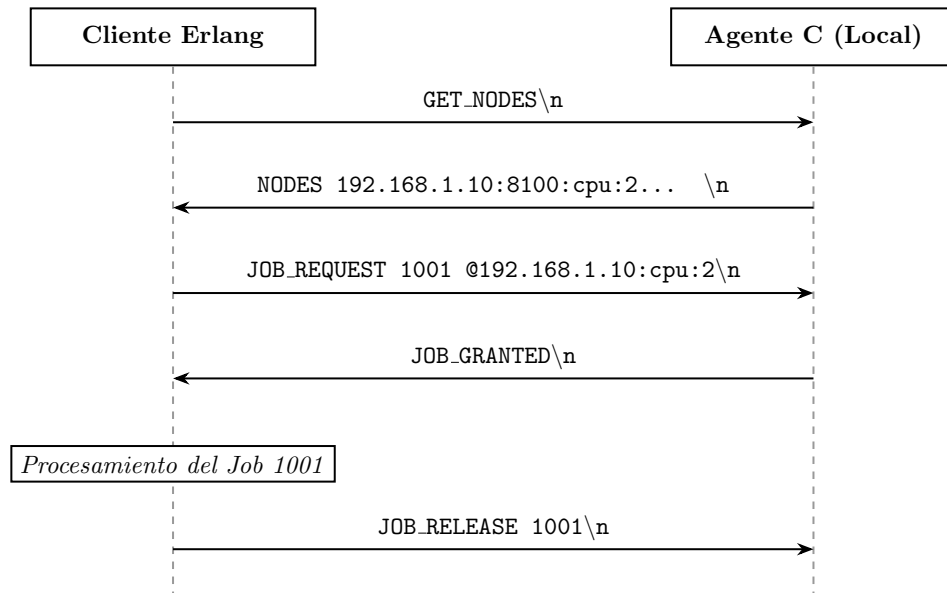
Si ambos planificadores en Erlang envían sus solicitudes en el mismo instante de tiempo, puede ocurrir la siguiente secuencia:

1. El Agente A recibe la petición de Job1 y entiende que puede ceder las 2 CPUs locales, por lo que las reserva exitosamente.
2. El Agente B recibe la petición de Job2 y sabe que puede ceder la GPU local, por lo que la reserva exitosamente.
3. El Agente A solicita la GPU al Agente B para Job1, pero como está retenida, la request se encola (quedando en espera). Simultáneamente, el Agente B solicita las CPUs al Agente A para Job2, quedando también encolada. Se ha formado una espera circular.
4. Ambos agentes detectan que sus solicitudes estuvieron en espera por un tiempo que excede un límite predefinido (timeout), lo que indica la presencia de un posible deadlock. Por lo tanto, ambos agentes envían un mensaje de timeout a sus respectivos planificadores en Erlang (Job1 y Job2).
5. Al recibir el error, los procesos de Job1 y Job2 en Erlang entran en un estado de suspensión por un tiempo aleatorio.
6. Debido a la varianza del tiempo de espera, un trabajo despertará antes que el otro (por ejemplo, Job1). Al reintentar su petición, como Job2 sigue suspendido y no retiene recursos, Job1 logra adquirir exitosamente las CPUs en A y la GPU en B. Cuando Job2 despierte, su petición se encolará hasta que Job1 finalice y libere la red, asegurando el progreso del sistema.

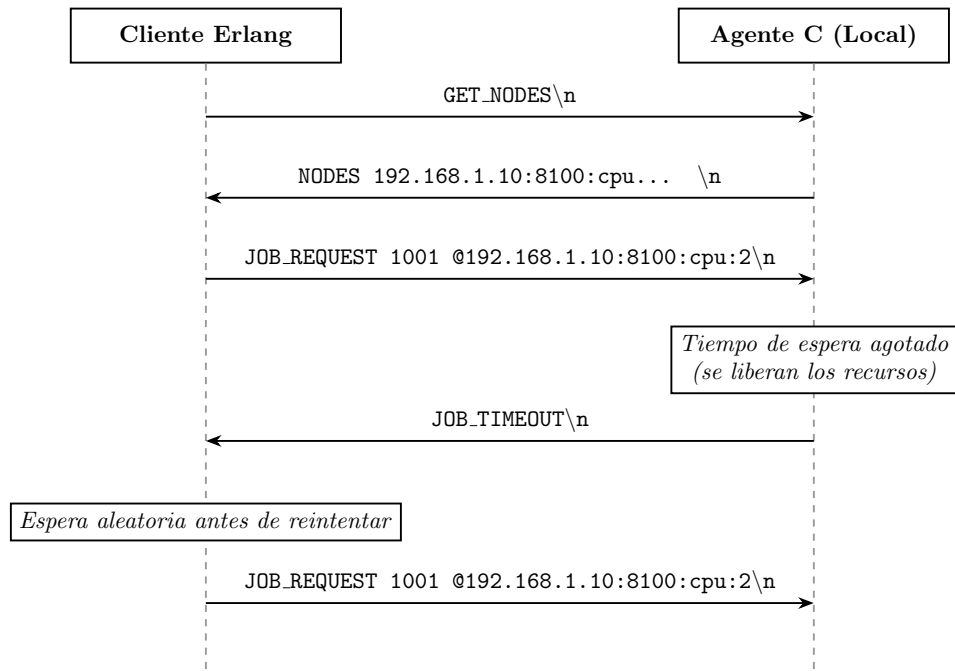
3 Diagramas de comunicación

3.1 Comunicación local (Cliente Erlang $\longleftrightarrow$ Agente C)

Caso GRANTED:

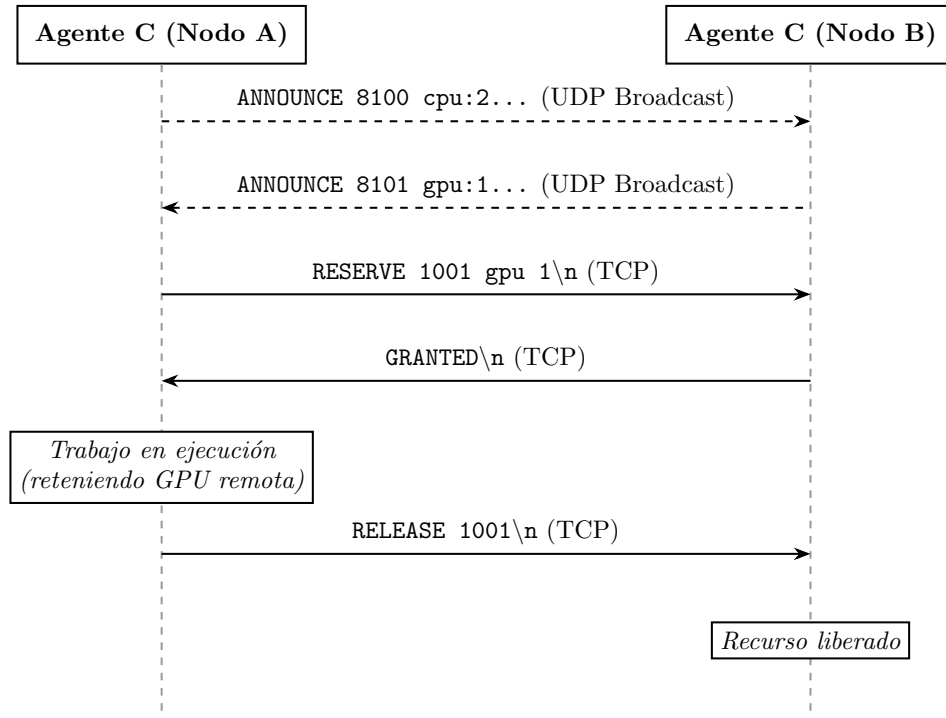


Caso TIMEOUT:



3.2 Comunicación entre nodos remotos

Secuencia de descubrimiento y reserva entre Nodos C:



Secuencia de comunicación para un job remoto (caso TIMEOUT):

